Temporal-Difference Learning

Richard S. Sutton and Andrew G. Barto, Reinforcement Learning: An Introduction (2nd Edition) Chapter6

유희철

2026-05-02

목차

1. TD Prediction
2. Advantages of TD Prediction Methods
3. Optimality
4. SARSA: On-Policy TD Control
5. Q-Learning: Off-Policy TD Control
6. Expected SARSA
7. Maximization Bias and Double Learning

TD Prediction

Monte Carlo에서는 이렇게 experience를 사용해서 Policy Prediction을 했음

$$V(S_t) \leftarrow V(S_t) + \alpha[G_t - V(S_t)]$$

→ Episode가 끝날 때까지 기다려야 함 (G_t를 알아야 하니)

TD Learning은 그럴 필요 없음

$$V(S_t) \leftarrow V(S_t) + \alpha[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$

1 Step만 기다리면 Update를 할 수 있음

TD(0) or one-step TD

TD Prediction

관측된 값 추정 값

$$V(S_t) \leftarrow V(S_t) + \alpha[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$

업데이트 할 값 현재 추정 값

Tabular TD(0) for estimating v_π

Input: the policy π to be evaluated
Algorithm parameter: step size $\alpha \in (0, 1]$
Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(terminal) = 0$

Loop for each episode:
  Initialize S
  Loop for each step of episode:
    $A \leftarrow$ action given by π for S
    Take action A, observe R, S'
    $V(S) \leftarrow V(S) + \alpha[R + \gamma V(S') - V(S)]$
    $S \leftarrow S'$
  until S is terminal

TD Prediction

Recall

$$
\begin{aligned}
v_\pi(s) &= \mathbb{E}[G_t | S_t = s] \\
&= \mathbb{E}[R_{t+1} + \gamma G_{t+1} | S_t = s] \\
&= \mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1}) | S_t = s]
\end{aligned}
$$

MC 방식은 초록색 식 기반의 추정치를 타겟으로 했고

DP 방식은 파란색 식 기반의 추정치를 타겟으로 했음

MC는 기댓값을 계산하지 않음 → 샘플링을 통해 하나의 샘플 반환 값(G_t) 을 사용해서 추정 (bootstrapping은 X)

DP는 기댓값을 직접 계산함 → 하지만 $v_\pi(S_{t+1})$를 모르기 때문에 추정값 $V(s')$ 을 사용

TD는 둘 다 사용 → 경험 기반의 샘플링 값(R_{t+1}, S_{t+1})를 사용하고 $V(S_{t+1})$를 추정함

TD Prediction

$$\text{MC Target} = G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ \ldots$$

$$\text{DP Target} = \sum_{s',r'} p(s',r'|s,\pi(s))[r + \gamma v_\pi(s')]$$

$$\text{TD Target} = R_{t+1} + \gamma V(S_{t+1})$$

TD는 MC의 model-free sampling과 DP의 bootstrapping을 합쳐 놓은 형태다

$$V(S_t) \leftarrow V(S_t) + \alpha[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$

TD Error

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$$

TD Prediction

$$
\begin{aligned}
G_t - V(S_t) &= R_{t+1} + \gamma G_{t+1} - V(S_t) + \gamma V(S_{t+1}) - \gamma V(S_{t+1}) \\
&= \delta_t + \gamma(G_{t+1} - V(S_{t+1})) \\
&= \delta_t + \gamma\delta_{t+1} + \gamma^2(G_{t+2} - V(S_{t+2})) \\
&= \delta_t + \gamma\delta_{t+1} + \gamma^2\delta_{t+2} + \cdots + \gamma^{T-t-1}\delta_{T-1} + \gamma^{T-t}(G_T - V(S_T)) \\
&= \delta_t + \gamma\delta_{t+1} + \gamma^2\delta_{t+2} + \cdots + \gamma^{T-t-1}\delta_{T-1} + \gamma^{T-t}(0 - 0) \\
&= \sum_{k=t}^{T-1} \gamma^{k-t}\delta_k
\end{aligned}
$$

MC Error는 TD Error의 Summation으로 나타날 수 있다. (단, V estimation이 다 끝났을 때)

Advantage of TD Prediction Methods

대충 뭐 MC랑 DP 장점만 섞어서 좋다는 얘기

- **DP에 비해서 TD는 어떻게 좋을까?**

TD는 DP와 다르게 환경 모델이 없어도 된다.

- **MC에 비해서 TD는 어떻게 좋을까?**

TD는 MC와 다르게 Online으로 target을 업데이트 할 수 있다.

Advantage of TD Prediction Methods

대충 뭐 MC랑 DP 장점만 섞어서 좋다는 얘기

- **DP에 비해서 TD는 어떻게 좋을까?**

TD는 DP와 다르게 환경 모델이 없어도 된다.

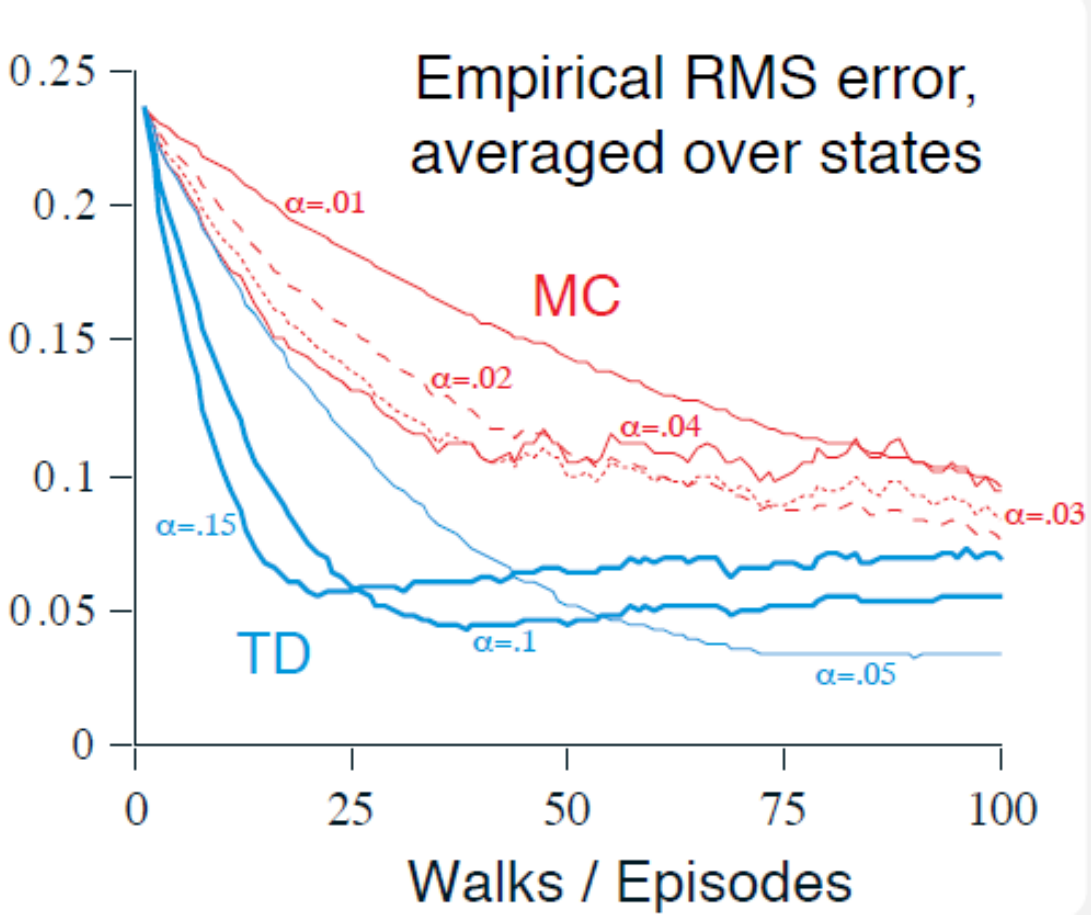
- **MC에 비해서 TD는 어떻게 좋을까?**

TD는 MC와 다르게 Online으로 target을 업데이트 할 수 있다.

→ 그럼 수렴하냐? *ans)* 예

→ 그럼 더 빠르냐? *ans)* 몰라

Advantage of TD Prediction Methods



Optimality of TD(0)

그럼 TD는 언제 MC보다 더 좋은걸까? 애초에 좋은게 맞을까?

→ 실제 Return을 확인하지 않고 추정 값으로 업데이트를 하는게 이게 정말 좋은 방식일까?

이 차이를 보기 위해 Batch Updating을 도입하자.

Batch Updating

- 지금까지 경험한 유한한 에피소드를 한 묶음의 batch로 만든다.
- Batch 안의 모든 transition에 대해서 TD update를 진행한다
- 단 Value function의 값을 바꾸지 말고 모든 increment를 모은다
- 다 모은 increment의 합으로 한 번에 갱신한다
- 같은 Batch를 다시 반복해서 처리한다
- Until Value Function이 변하지 않을 때 까지

Optimality of TD(0)

Batch MC

각 상태를 방문한 뒤 실제로 얻은 Return의 평균으로 수렴

$$V(s) = \frac{0 + 1 + 1}{3} = \frac{2}{3}$$

경험 안의 실제 Return들에 대한 MSE를 최소화하는 값으로 수렴

$$\min_V \sum_s (G_t - V(s))^2$$

MC는 실제로 경험에서 나타난 Return들을 가장 잘 맞추는 값으로 수렴

Optimality of TD(0)

Batch TD(0)

실제 Return들의 평균으로 수렴하지 않는다. TD는 아래 관계식을 맞추도록 학습됨

$$\boldsymbol{V(S_t) \sim R_{t+1} + \gamma V(S_{t+1})}$$

e.g.) 다음 2개의 에피소드가 있음

$$A \rightarrow B$$
$$A \rightarrow C$$

그럼 Batch TD로 $V(A)$를 업데이트하려면 다음과 같이 됨

$$\boldsymbol{V(S_t) = \frac{1}{2}[r_{A\rightarrow B} + \gamma V(B)] + \frac{1}{2}[r_{A\rightarrow C} + \gamma V(C)]}$$

즉, Batch TD는 관측된 Transition 구조가 만드는 Bellman Equation의 해를 맞추도록 학습

Optimality of TD(0)

MC : 현재 상태에서 시작해서 실제로 최종 결과가 어땠는지를 평균 냄

TD : 현재 상태는 다음 상태들과 어떤 관계를 갖는지를 확인

데이터가 적고 Stochastic한 상황에서 TD가 더 잘 일반화할 수 있다.

- Batch TD는 관측된 데이터로부터 maximum-likelihood Markov Model을 만들고 모델에 대한 정확한 value function을 계산

SARSA: On-Policy TD Control

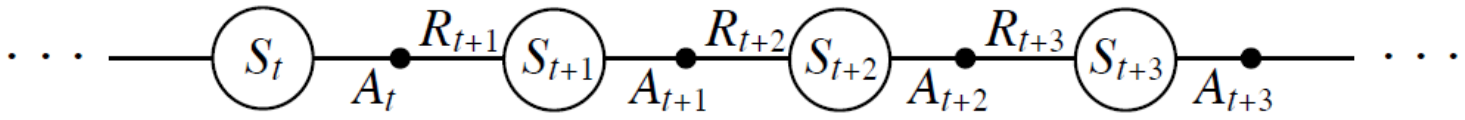
이제 Control Side로 넘어가자

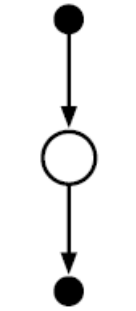
먼저 On-Policy TD Control부터.

Control 문제에서는 어떤 행동을 선택해야 할 지 알아야 하기 때문에 행동가치함수로 업데이트

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$$

A_{t+1} 와 A_t는 정책 π에 따라 선택된 행동 (On-Policy)





Sarsa

SARSA: On-Policy TD Control

$$
\begin{aligned}
G_t - Q(S_t, A_t) &= R_{t+1} + \gamma G_{t+1} - Q(S_t, A_t) + \gamma Q(S_{t+1}, A_{t+1}) - \gamma Q(S_{t+1}, A_{t+1}) \\
&= \delta_t + \gamma(G_{t+1} - Q(S_{t+1}, A_{t+1})) \\
&= \delta_t + \gamma\delta_{t+1} + \gamma^2(G_{t+2} - Q(S_{t+2}, A_{t+2})) \\
&= \delta_t + \gamma\delta_{t+1} + \gamma^2\delta_{t+2} + \cdots + \gamma^{T-t-1}\delta_{T-1} + \gamma^{T-t}(G_T - Q(S_T, A_T)) \\
&= \sum_{k=t}^{T-1} \gamma^{k-t}\delta_k
\end{aligned}
$$

State-Value Function으로 유도했던 TD error summation을 State-Action Value Function으로도 유도 가능

Q-Learning: Off-Policy TD Control

이젠 Off-Policy로 TD Control을 해보자

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma \max Q(S_{t+1}, a) - Q(S_t, A_t)]$$

$\max Q(S_{t+1}, a)$를 통해 q_*를 바로 학습하려 하는 것을 확인할 수 있음 (Not q_π)

→ 즉 polic가 어떻든 관계 없이 최적 행동가치함수를 근사하고 있음 (Off-Policy)

다음 State로의 행동은 정책대로 하더라도 (다음 S_{t+1} 를 고르는 일)

다음 State의 $Q(S_{t+1}, a)$ 는 최적 행동가치함수를 배우도록 학습 목표를 잡음

→ SARSA의 경우 다음 State의 $Q(S_{t+1}, A_{t+1})$ 를 구하기 위해 현재 정책 π를 따라서 행동을 결정했음

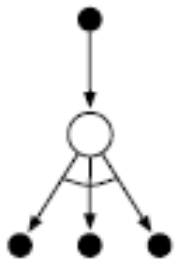
Q-Learning은 행동은 exploration 정책으로 하고 학습은 greedy 최적 정책 기준으로 함

Q-Learning: Off-Policy TD Control

Q-learning (off-policy TD control) for estimating $\pi \approx \pi_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$
Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, arbitrarily except that $Q(terminal, \cdot) = 0$

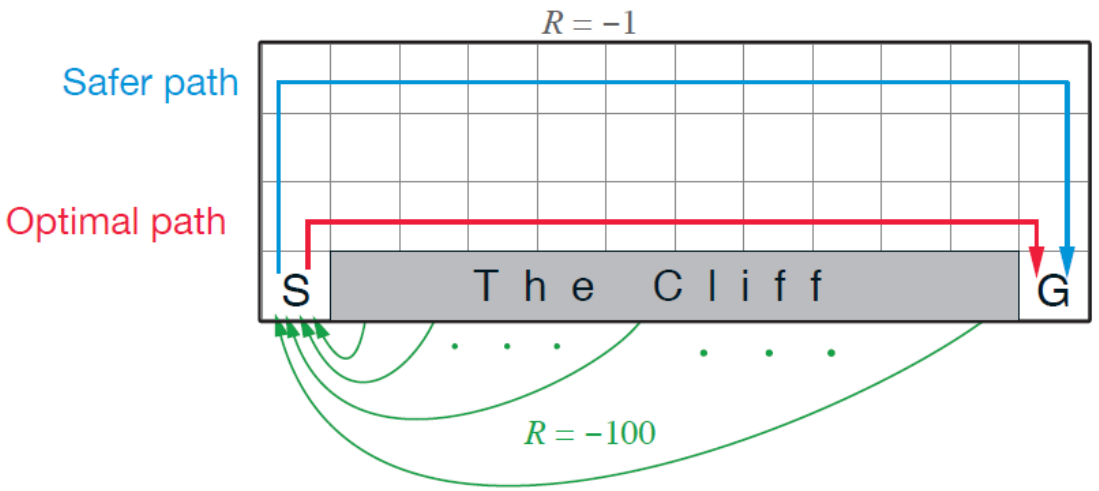
Loop for each episode:
  Initialize S
  Loop for each step of episode:
    Choose A from S using policy derived from Q (e.g., ε-greedy)
    Take action A, observe R, S'
    $Q(S, A) \leftarrow Q(S, A) + \alpha\big[R + \gamma \max_a Q(S', a) - Q(S, A)\big]$
    $S \leftarrow S'$
  until S is terminal



Q-learning

Q-Learning: Off-Policy TD Control

Example: Cliff Walking



Action : Up / Down / Left / Right

Reward : 한 칸 이동 R = -1, 절벽에서 떨어지면 R = -100 (and go to S)

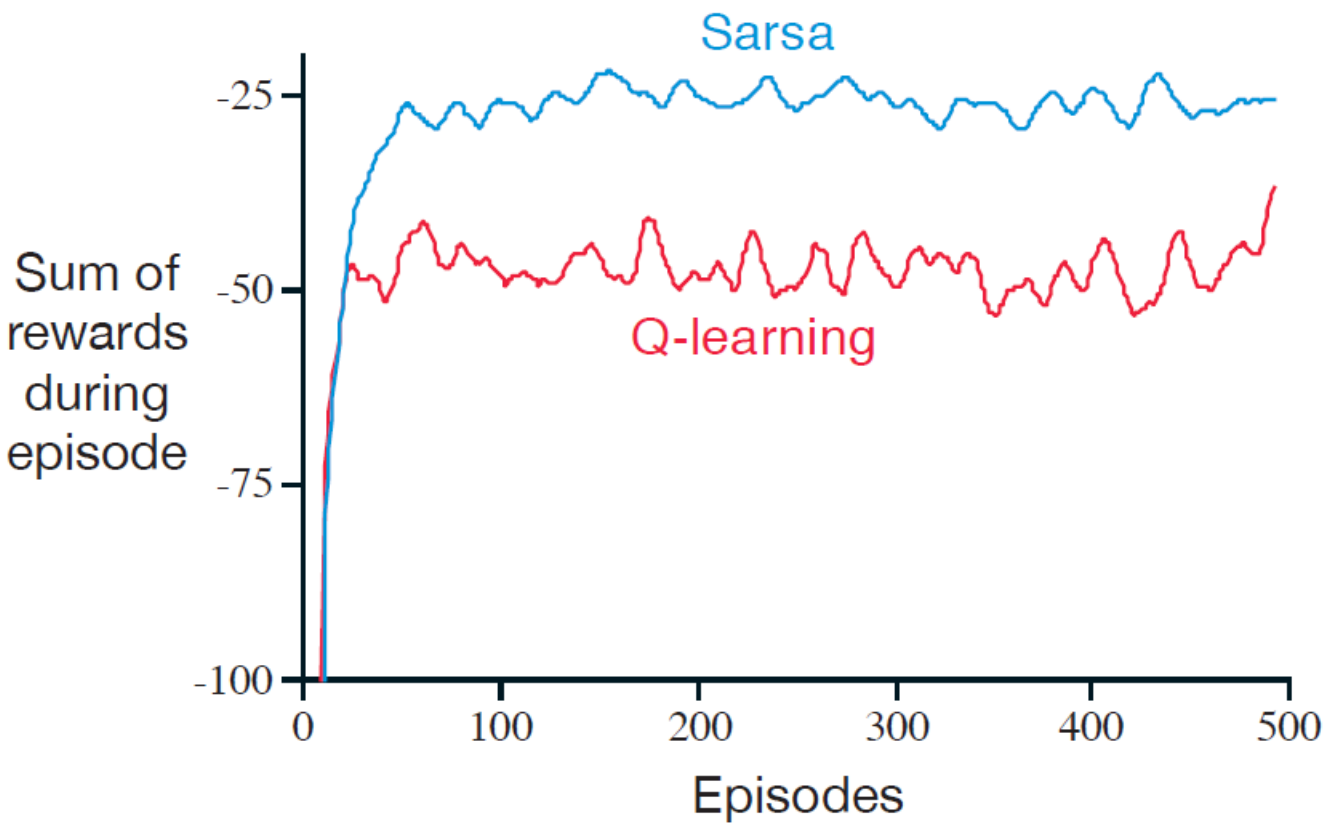
ϵ-greedy 정책 때문에 랜덤 행동을 하다 아래로 움직이면 떨어져서 Optimal Path는 사실 좀 위험함

근데? Q-Learning은 이 Optimal Path를 배우도록 설계되어 있지

Q-Learning: Off-Policy TD Control

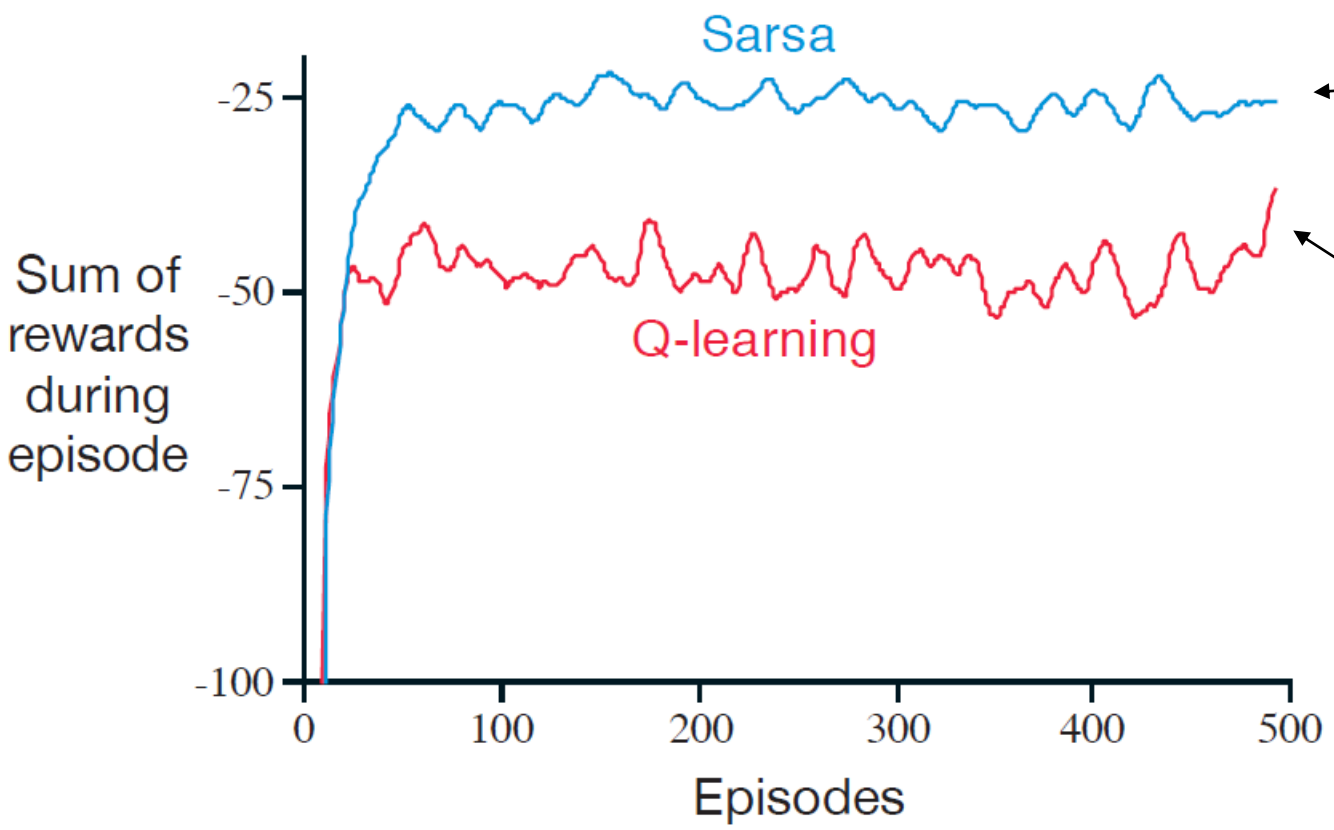
Example: Cliff Walking

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma \max Q(S_{t+1}, a) - Q(S_t, A_t)]$$



Q-Learning: Off-Policy TD Control

Example: Cliff Walking

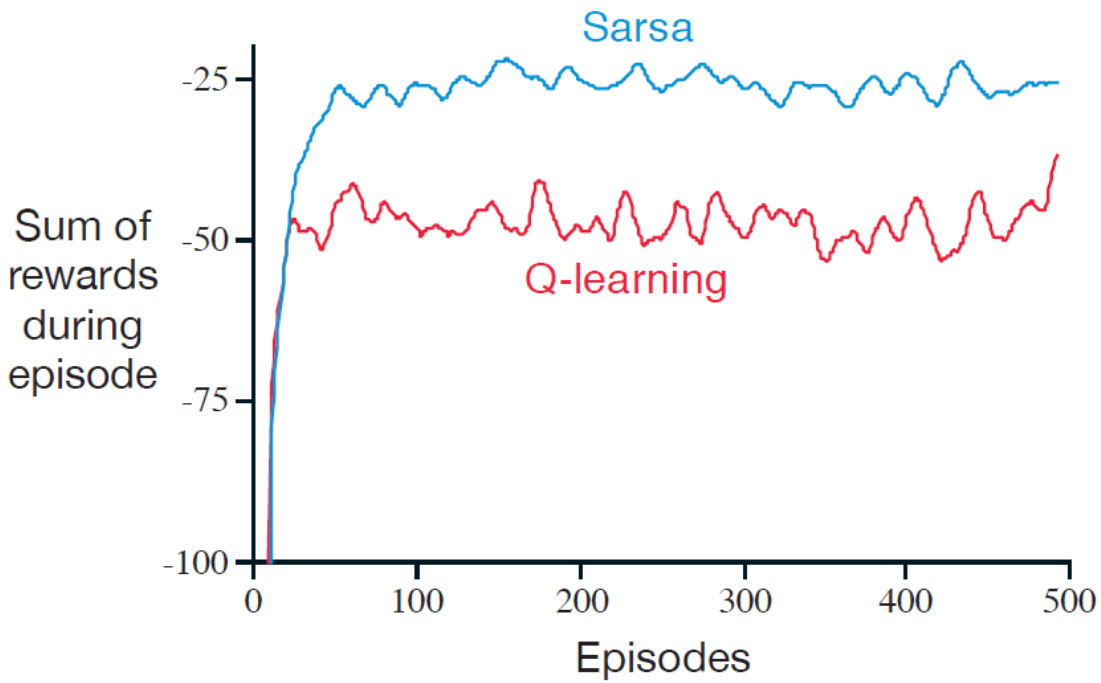


SARSA 수행 중 선택하는 A_{t+1}에도 역시 ϵ-greedy 정책이 선택됨
이 실수까지 고려해서 가치 평가

실제 수행 중 자주 절벽으로 떨어지기 때문에 Q-Learning은 Reward가 나쁨

Q-Learning: Off-Policy TD Control

Example: Cliff Walking



SARSA는 Exploration으로 인한 risk까지 반영한다

Q-Learning은 Exploration으로 인한 risk를 반영하지 않는다

물론 ϵ 이 점점 줄어들면 두 방법은 모두 점근적으로 최적에 수렴 ($\epsilon \rightarrow 0$이면 탐험을 안하니까)

Expected SARSA

SARSA는 실제로 뽑힌 정책에 따른 행동 하나를 사용해서 Target을 계산했음

$$R_{t+1} + \gamma Q(S_{t+1}, A_{t+1})$$

Q-Learning은 다음 행동 중 가장 큰 하나의 optimal 값을 사용해서 Target을 계산했음

$$R_{t+1} + \gamma \max Q(S_{t+1}, a)$$

Expected SARSA는 SARSA처럼 현재 정책의 위험을 반영하는 동시에 샘플링된 행동 하나에 의존하지 않도록 설계함

$$R_{t+1} + \gamma \sum_{a} \pi(a|S_{t+1}) Q\,(S_{t+1}, a)$$

정책이 고를 수 있는 모든 행동 가치의 기댓값을 계산하고 이를 Target을 계산하는데 사용함

Expected SARSA

즉 다음과 같이 Control

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma \mathbb{E}[Q(S_{t+1}, A_{t+1}) | S_{t+1}] - Q(S_t, A_t)]$$

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma \sum_a \pi(a|S_{t+1}) Q(S_{t+1}, a) - Q(S_t, A_t)]$$

Expected SARSA에서 행동을 생성하기 위해 타겟 정책인 π와 다른 정책을 사용할 수도 있음

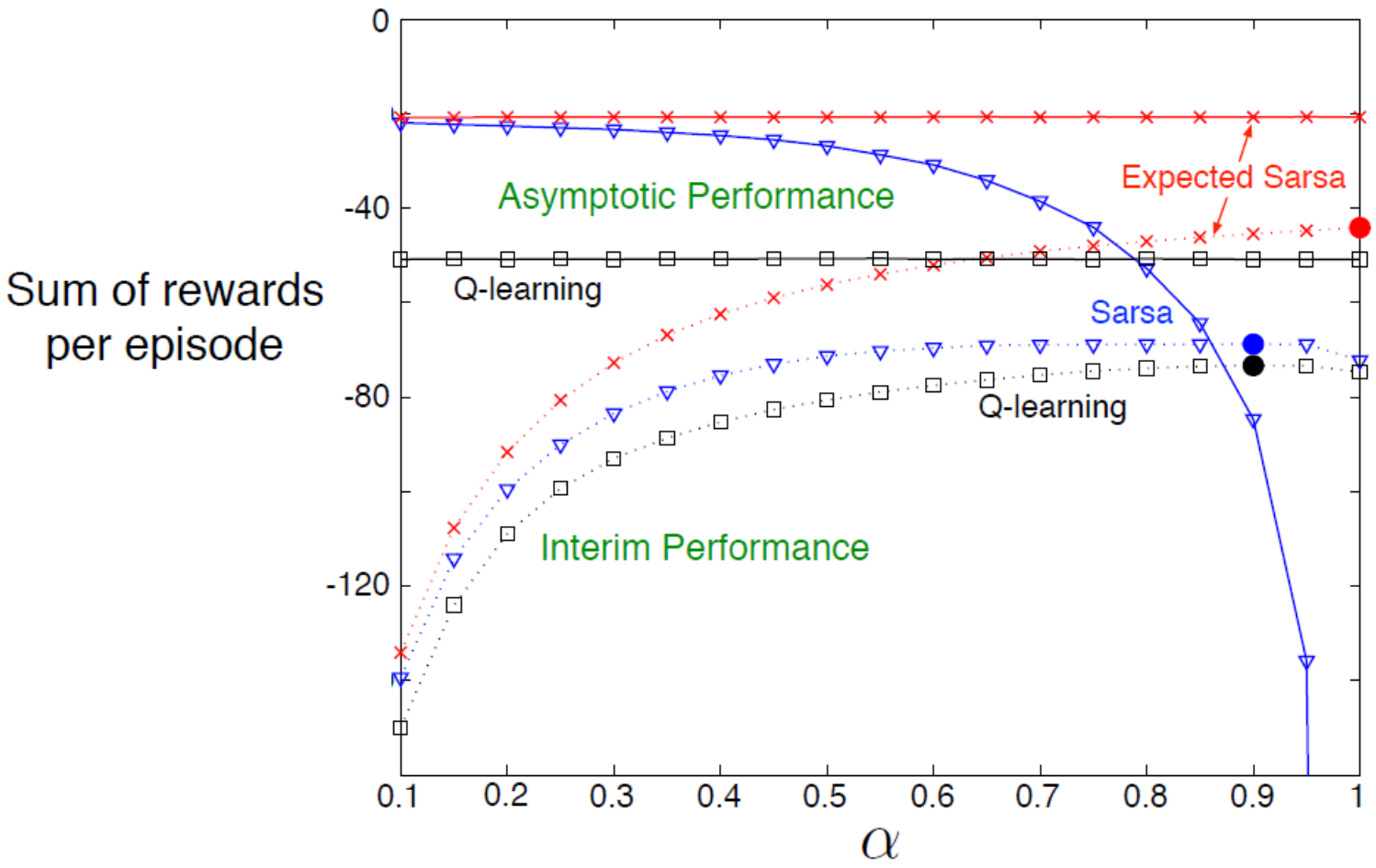
→ 이러면 Off-Policy 알고리즘이 되겠지

Expected SARSA는 Q-Learning을 포함하고 일반화한 개념이자 SARSA보다 개선된 성능을 보임

SARSA처럼 현재 정책의 위험을 반영하는 동시에 행동 하나에 의존하지 않아 분산이 작아서 안정적임

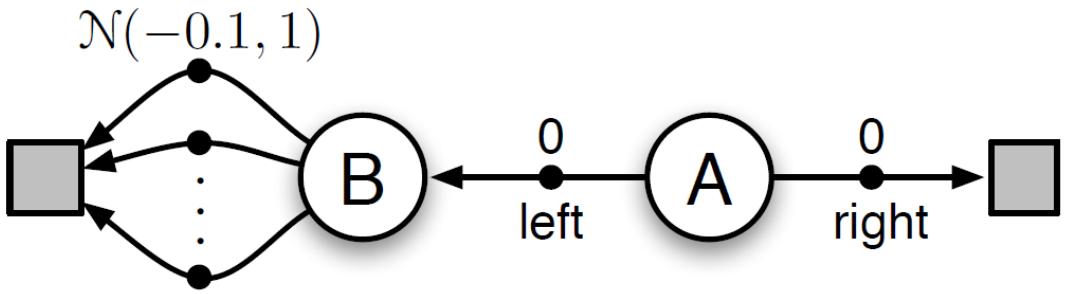
Expected SARSA

Example: Cliff Walking



Maximization Bias and Double Learning

Example: Maximization Bias Example



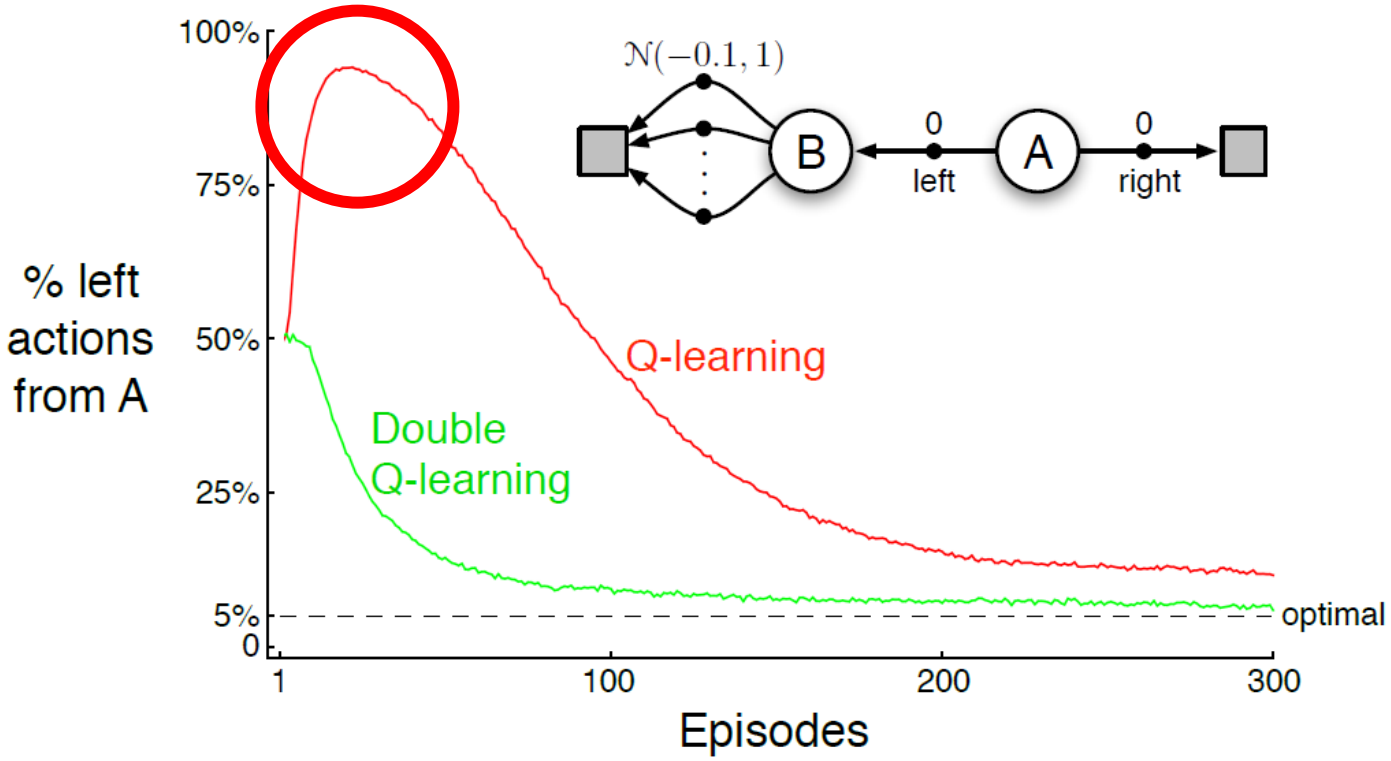
State A에서 출발, Action은 Left와 Right. 이 두 Action의 보상은 0.

- Right을 선택하면 바로 Terminal State
- Left를 선택하면 State B로 이동. B에선 평균이 -0.1이고 분산이 1.0인 정규분포를 따르는 보상을 받고 terminal로 이동
- Left를 선택하면 기대보상은 -0.1이 되겠지? 그럼 Right로 가야겠구나? (기대보상이 0이니까)

하지만 우리가 설계한 control method는 left를 선택하는 걸 더 선호함

Maximization Bias and Double Learning

Example: Maximization Bias Example



Maximization Bias

Maximization Bias and Double Learning

Maximization Bias

우리는 Q-Learning Target을 다음과 같이 잡았었음

$$R_{t+1} + \gamma \max Q(S_{t+1}, a)$$

근데 Q 값들의 분산이 크다면? 실제로 좋은 행동이 아니라 우연히 과대평가된 행동이 선택될 수도 있음

→ Maximum Bias

그럼 이 편향 문제를 피할 수 있을까? 문제를 두 개로 쪼개보자.

1. 어떤 행동이 최대값을 가지는지 확인 (행동 선택)
2. 그 최대값을 가지는 행동의 가치를 추정 (가치 평가)

Maximization Bias and Double Learning

1. 어떤 행동이 최대값을 가지는지 확인 (행동 선택) → Q_1
2. 그 최대값을 가지는 행동의 가치를 추정 (가치 평가) → Q_2

Double Q-Learning에서는 위 두 가지에 대해 각각 다른 Q를 도입해서 그 둘을 따로 추정함

$$\mathbf{argmax}\ Q_1(S_{t+1}, a)$$

Q_1으로 어떤 행동이 좋아 보이는지 선택하고

$$Q_2(S_{t+1}, \mathbf{argmax}\ Q_1(S_{t+1}, a))$$

이 행동의 실제 TD Target Value는 Q_2로 평가 (Not MAX!)

이 추정 값은 편향되지 않음. 왜냐? $\mathbb{E}[Q_2(A^*)] = q(A^*)$

Maximization Bias and Double Learning

Double Q-learning, for estimating $Q_1 \approx Q_2 \approx q_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$
Initialize $Q_1(s, a)$ and $Q_2(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, such that $Q(terminal, \cdot) = 0$

Loop for each episode:
  Initialize S
  Loop for each step of episode:
    Choose A from S using the policy ε-greedy in $Q_1 + Q_2$
    Take action A, observe R, S'
    With 0.5 probabilility:

$$Q_1(S, A) \leftarrow Q_1(S, A) + \alpha\Big(R + \gamma Q_2\big(S', \arg\max_a Q_1(S', a)\big) - Q_1(S, A)\Big)$$

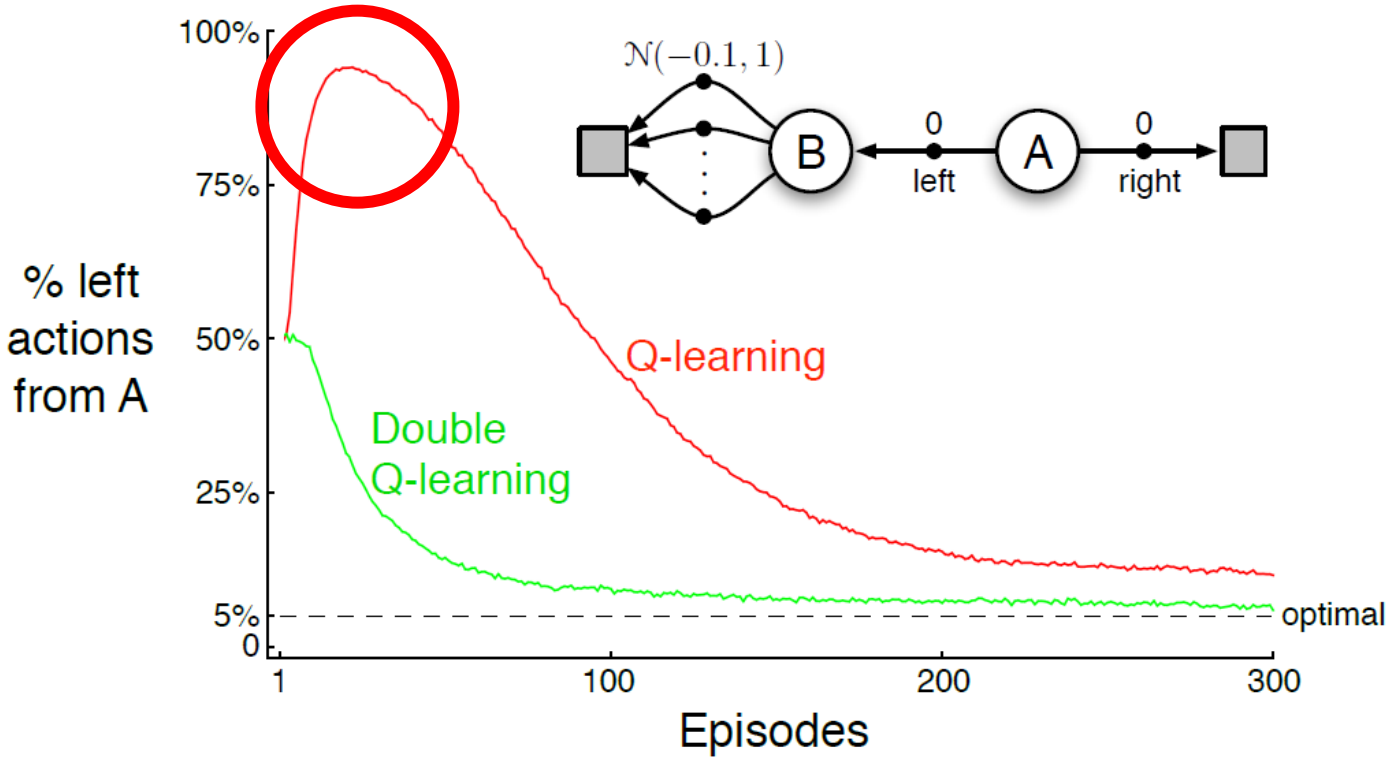
    else:

$$Q_2(S, A) \leftarrow Q_2(S, A) + \alpha\Big(R + \gamma Q_1\big(S', \arg\max_a Q_2(S', a)\big) - Q_2(S, A)\Big)$$

    $S \leftarrow S'$
  until S is terminal

Maximization Bias and Double Learning

Example: Maximization Bias Example



Maximization Bias

Maximization Bias and Double Learning

Double Expected SARSA

$$Q_1(S_t, A_t) \leftarrow Q_1(S_t, A_t) + \alpha[R_{t+1} + \gamma \sum_a \pi(a|S_{t+1}) Q_2(S_{t+1}, a) - Q_1(S_t, A_t)]$$

$$\pi(a|s) = \begin{cases} 1 - \epsilon + \dfrac{\epsilon}{|A(s)|} & \text{if } a = \text{argmax } Q_1(s, a') + Q_2(s, a') \\ \dfrac{\epsilon}{|A(s)|} & \end{cases}$$

Q & A